

Scalable AI

Scaling Laws

Lorenzo Baraldi

Outline

- **Scaling Laws**
 - Kaplan et al., 2020 (OpenAI)
 - Kaplan et al., 2021 (OpenAI – Scaling for Transfer)
 - Hutter, 2021 (DeepMind)
 - Hoffman et al., 2022 (DeepMind)
- **Emergent Abilities**
 - Wei et al., 2022
 - Schaeffer et al., 2023
- **Complicating Scaling Laws**
 - Wei et al., 2022 (U-Shaped)
 - Tay et al., 2022

Motivation

- Scientific problem: How does model performance change with increasing scale?
- Engineering importance: Training frontier models costs millions of dollars
- Safety importance: Models are black boxes with often unexpected capabilities

Scaling Laws for Neural Language Models, Kaplan et al. 2020

Main RQ: What is the relationship between compute, data, the number of parameters, and performance?

Kaplan et al., 2020 - Main Idea

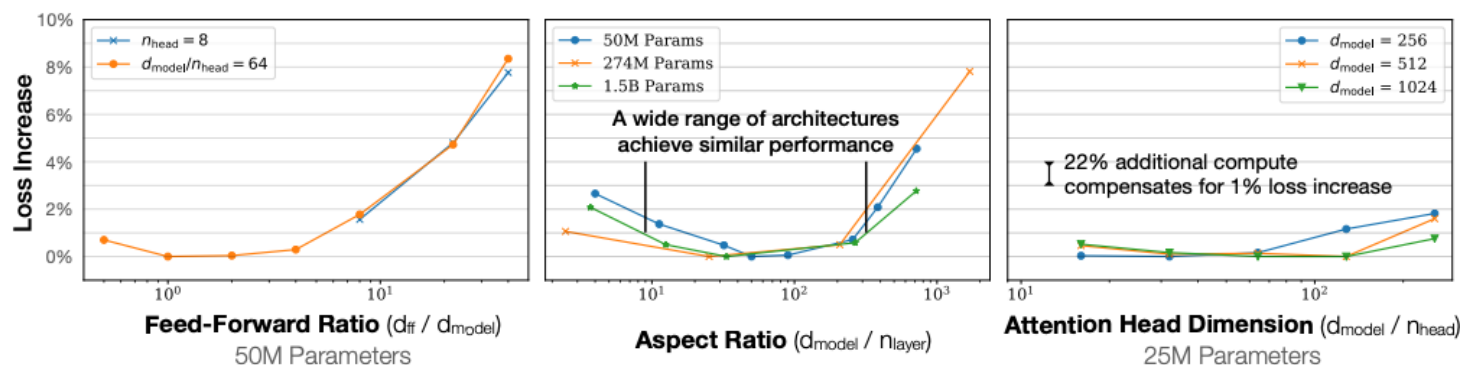
- Train models of vastly different scale (data, compute, params) and find the loss is predictably related to the scale
- Offer a predictive framework
- They make certain recommendations based on the scaling laws
 - Ratio between parameters and data
 - Don't train to convergence
 - Critical batch size

Kaplan et al., 2020 - Main Results

1. Performance depends strongly on scale, weakly on model shape
2. Performance scales with each of compute (C), data (D), and # of parameters (N) when not bottlenecked by the other two
3. Performance improves predictably if we scale N and D in tandem, but suffers diminishing returns if N or D is held fixed
4. Training curves are predictable, so we can (roughly) predict the final loss by extrapolating the early part of the training curve, regardless(ish) of model size
5. We can predict how well the model will perform on OOD data by looking at the training validation accuracy
6. Large models are more sample efficient than small ones
7. Convergence is inefficient!
8. The ideal batch size (B_{crit}) is roughly a power of the loss only

Performance mostly depends on scale

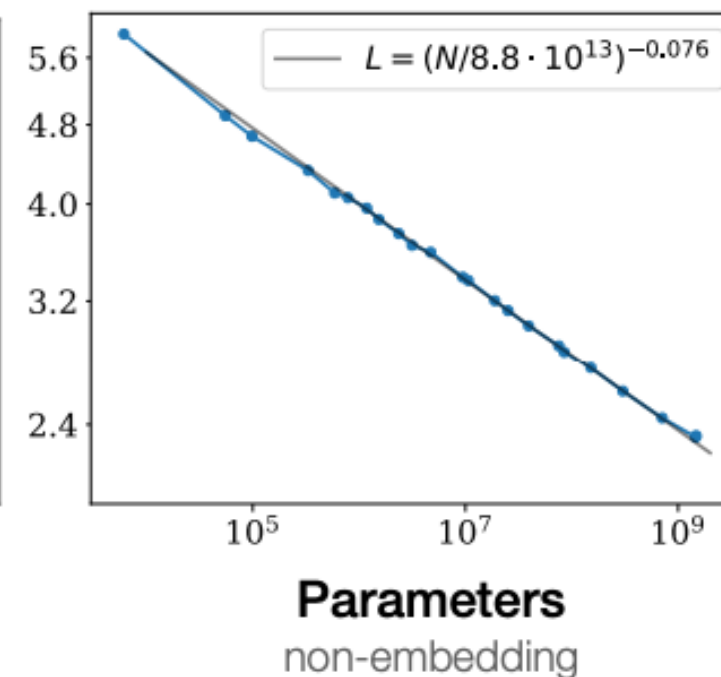
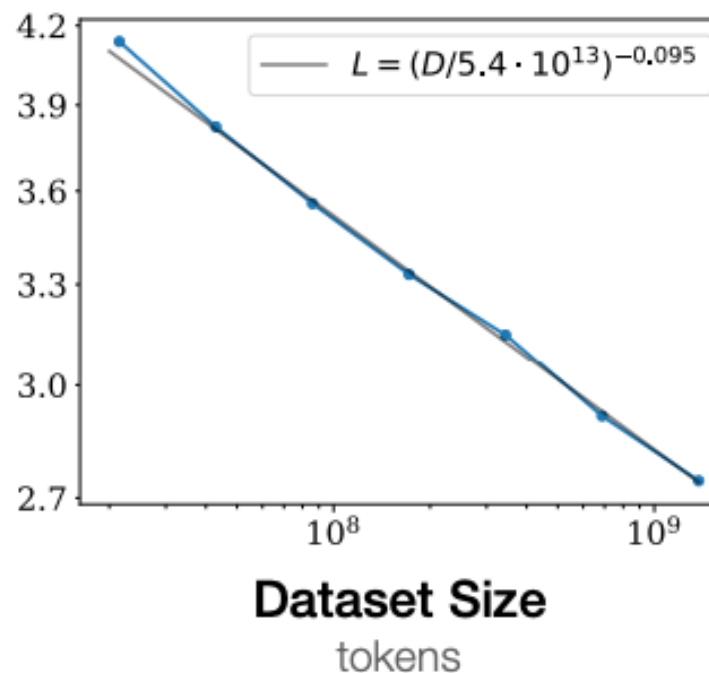
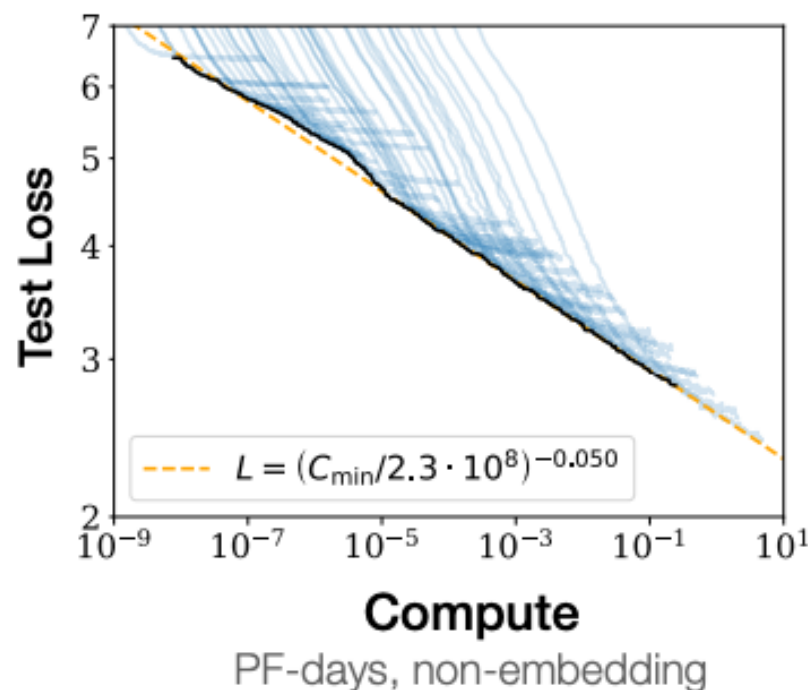
- d_{model} : dimension of residual stream
- d_{ff} : dimension of intermediate feed forward layer
- n_{layer} : number of layers
- n_{heads} : number of attention heads per layer
- Hold total non-embedding parameter count N fixed
 - "This was simplest for the case of n_{heads} " (right)
 - When varying n_{layer} , we simultaneously varied d_{model} while keeping $N \approx 12n_{layer}d_{model}^2$ fixed (middle)
 - Similarly, to vary d_{ff} at fixed model size we also simultaneously varied the d_{model} parameter (left)



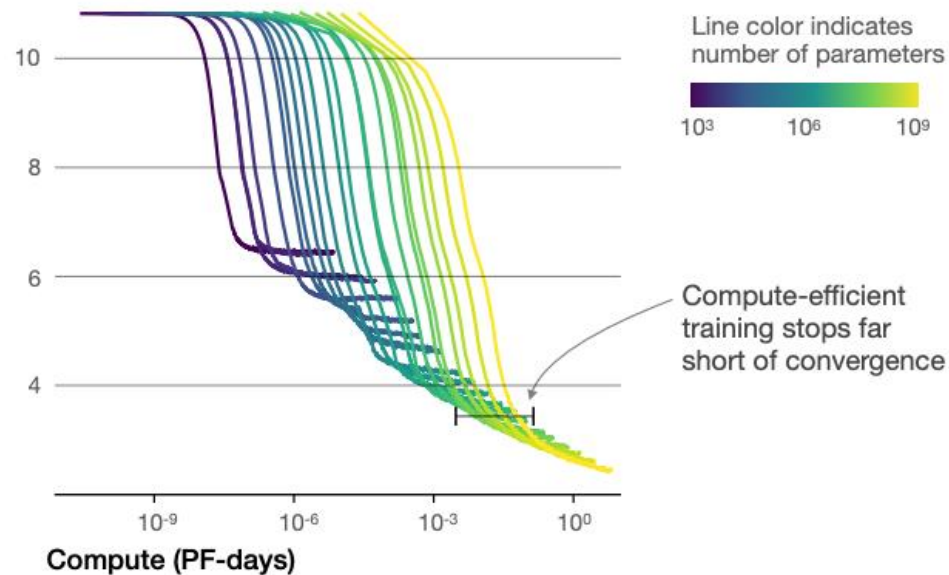
Kaplan et al., 2020

- A Power Law: $y = (cx)^a$
- Linear on a log scale: $\ln(y) = a \ln(x) + a \ln(c)$

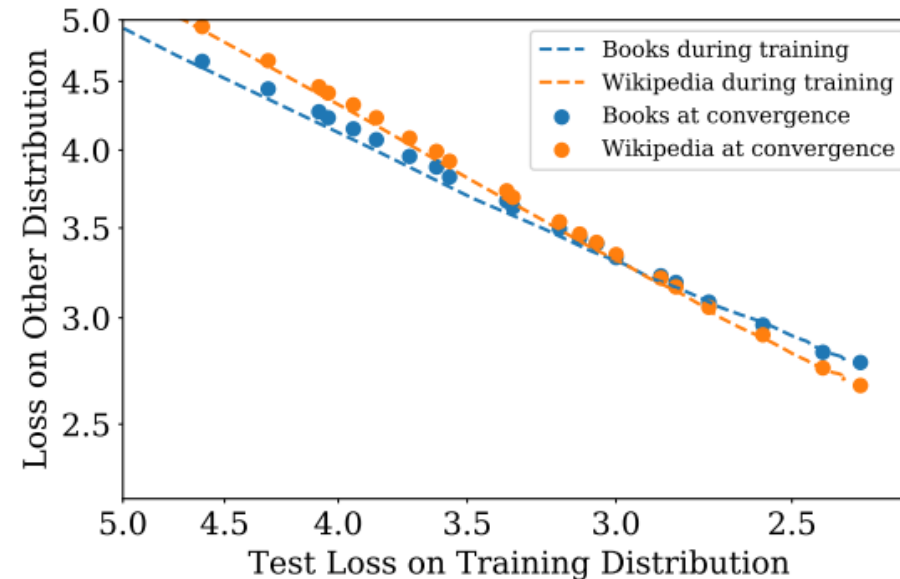
- Performance has a power-law relationship with each of the three scale factors N, D, C when not bottlenecked by the other two, with trends spanning more than six orders of magnitude



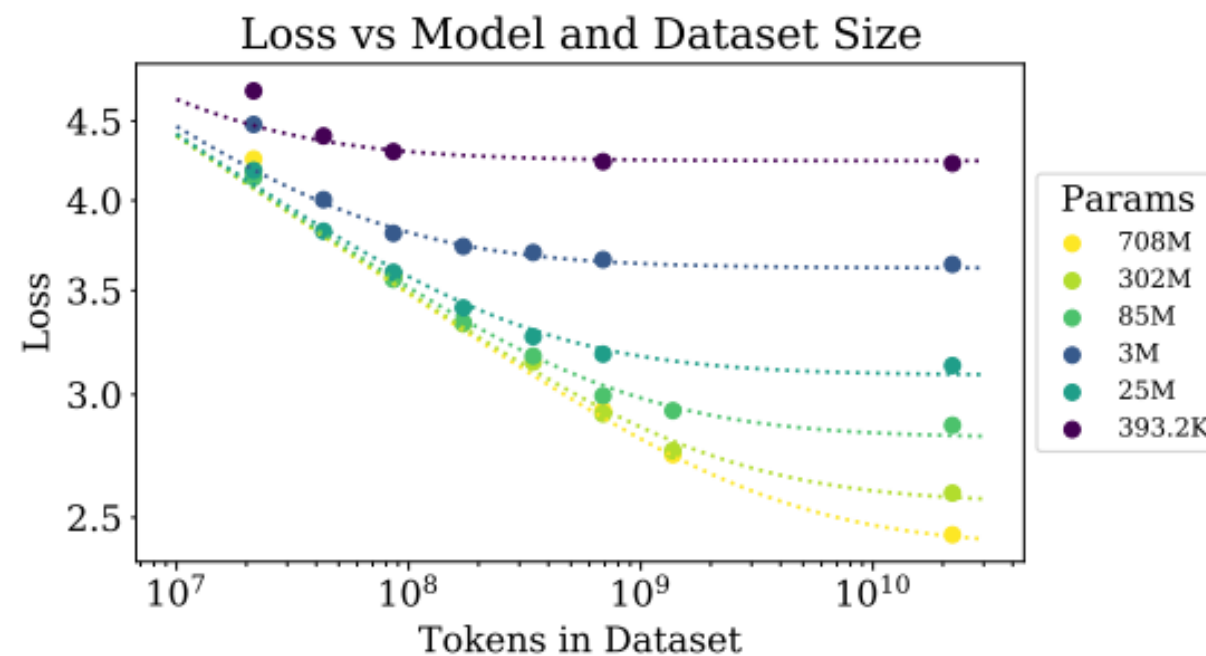
- Training curves follow predictable power-laws whose parameters are roughly independent of the model size.
- By extrapolating the early part of a training curve, we can roughly predict the loss that would be achieved if we trained for much longer.



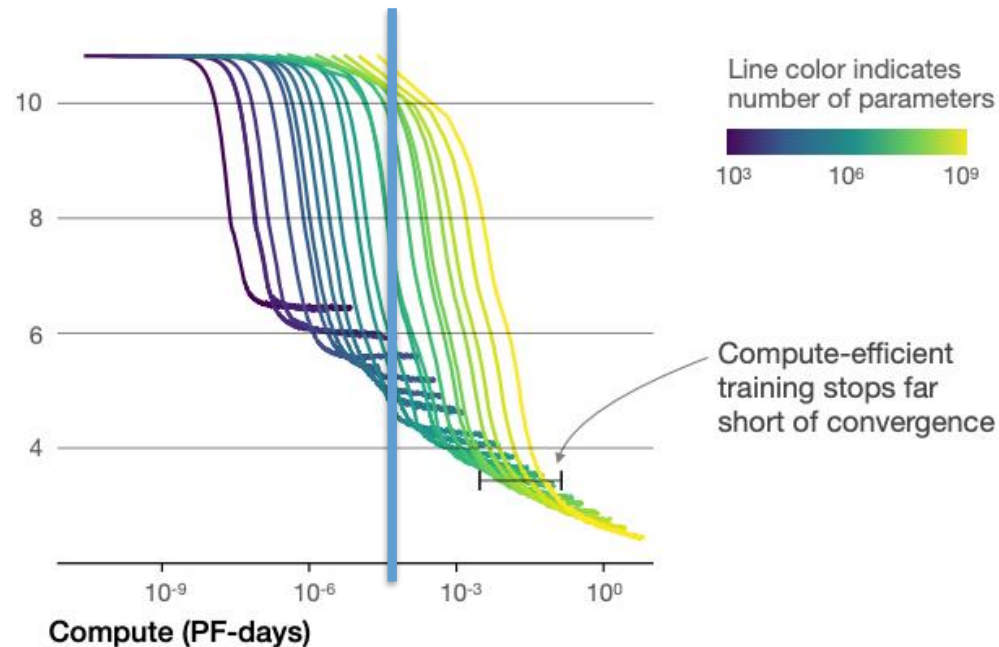
- When we evaluate models on text with a different distribution than they were trained on, the results are strongly correlated to those on the training validation set with a roughly constant offset in the loss



- Large models are more sample-efficient than small models, reaching the same level of performance with fewer optimization steps (Left) and using fewer data points (Right).



- Convergence is inefficient
- When working within a fixed compute budget C but without any other restrictions on the model size N or available data D , we attain optimal performance by training very large models and stopping significantly short of convergence.

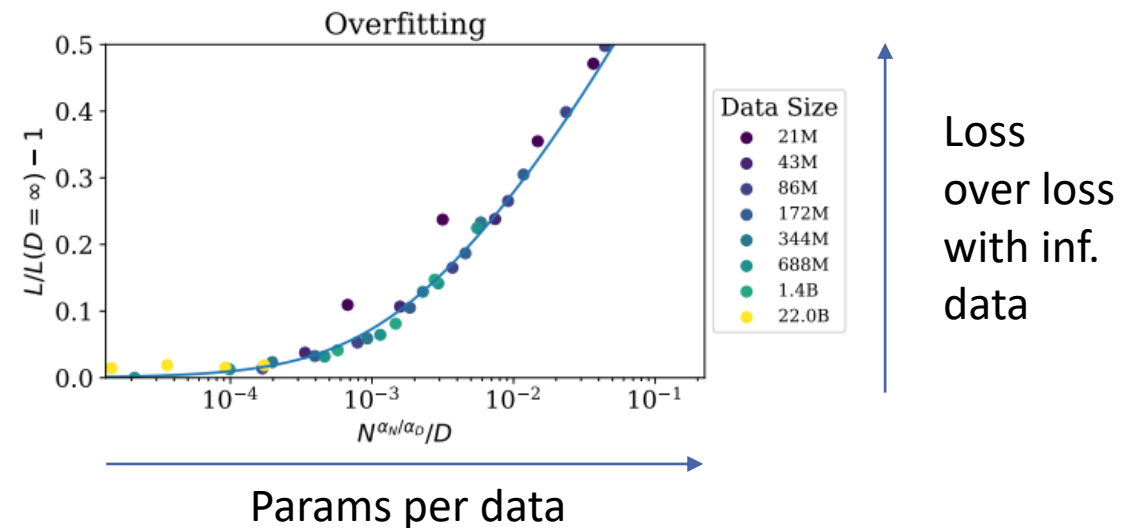
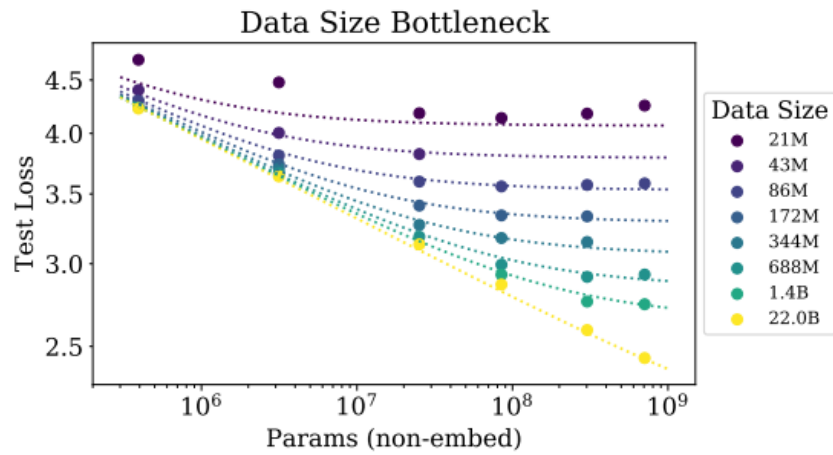


For example, imagine you have 10^{-4} PF-days of compute

Smaller model trained to convergence achieve a loss of ~ 6

Larger models achieve a loss < 5

- Performance improves predictably as long as we scale up N and D in tandem, but enters a regime of diminishing returns if either N or D is held fixed while the other increases.
- The performance penalty depends predictably on the ratio $N^{0.74}/D$
 - every time we increase the model size 8x, we only need to increase the data by roughly 5x to avoid a penalty.



- The ideal batch size for training these models is roughly a power of the loss only, and continues to be determinable by measuring the gradient noise scale [MKAT18]; it is roughly 1-2 million tokens at convergence for the largest models we can train. (Section 5.1)

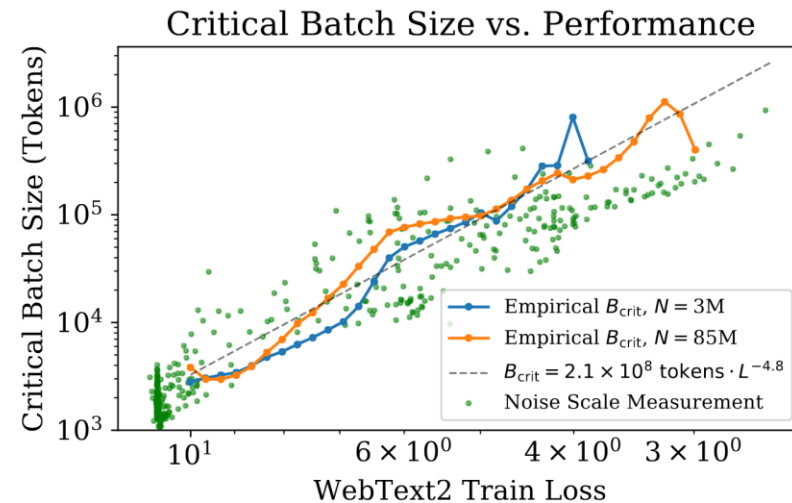


Figure 10 The critical batch size B_{crit} follows a power law in the loss as performance increase, and does not depend directly on the model size. We find that the critical batch size approximately doubles for every 13% decrease in loss. B_{crit} is measured empirically from the data shown in Figure 18, but it is also roughly predicted by the gradient noise scale, as in [MKAT18].

- Takeaways
 - Cross-entropy loss scales predictably with model scale
 - Model shape matters far less than scale
 - The amount of embedding parameters doesn't really matter
 - We can predict the final loss of a model based on the early part of the training curve
 - Training models to convergence is compute inefficient
 - The ideal batch size is a power of loss only and does not depend on model size

Scaling Laws for Transfer, Kaplan et. al. 2021



Main RQ: What are the empirical scaling laws for transfer learning?

Kaplan et. al. 2021 – Main Results

- Pretraining is very valuable in the low-data regime
- Quantify "effective data transferred" with (yet) another power law
- Conjectured measurement of similarity between datasets
- Investigate "ossification": Can pretraining hurt performance?

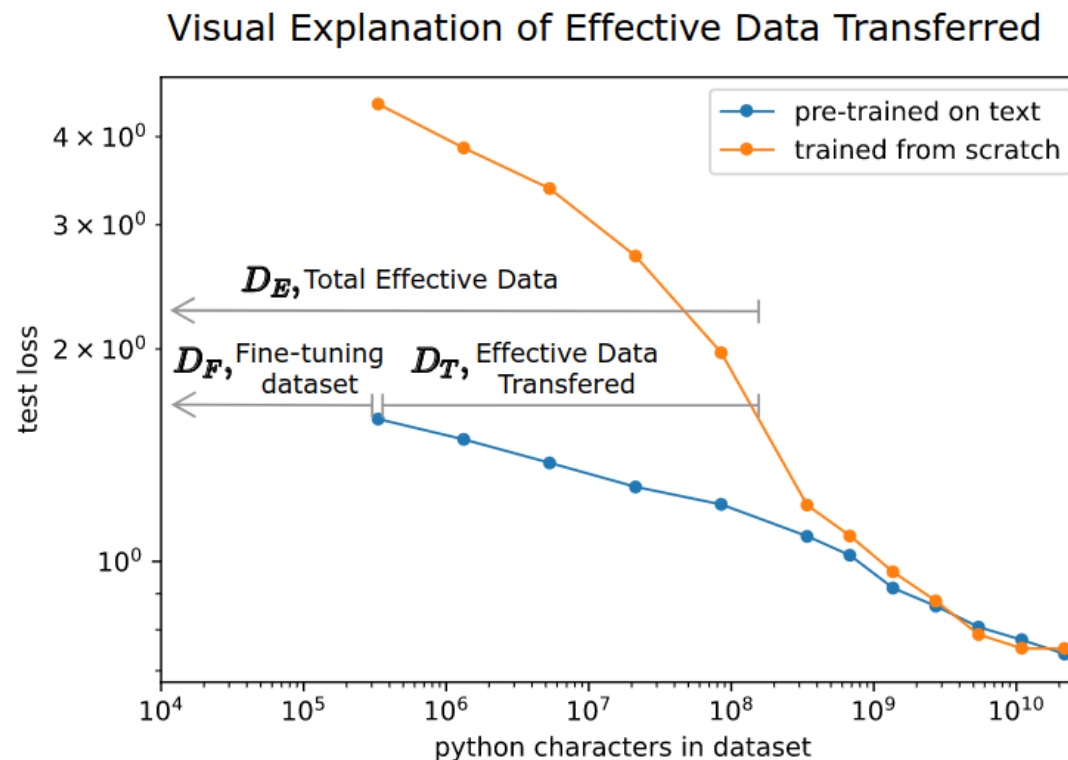


Figure 1 We display the performance of a 40M parameter transformer model on python, both trained from scratch on python and pre-trained on text then fine-tuned on python. D_T is the amount of additional python characters that a from-scratch model of the same size would have needed to achieve the same loss on python as a fine-tuned model. In the labeled example, we see that for a 40M parameter transformer fine-tuned on 3×10^5 characters, D_T is approximately 1000x bigger than D_F . The less fine-tuning data is available, the more pre-training helps.

D_F = Fine tuning data

N = non-embedding model parameters

$$D_T = \text{effective data transferred} = k(D_F)^\alpha(N)^\beta$$

Transfer from	Transfer Coefficients		
	k	α	β
Text $\implies$ Python	1.9e4	0.18	0.38
50% Text and 50% non-python code $\implies$ Python	2.1e5	0.096	0.38

Table 1 Summary of transfer coefficients. The larger k value indicates that mixture models transfer more readily than plain text in the low-data regime, while the smaller α means the benefit diminishes as we approach the high-data regime.

- "Ossification": when pre-training hurts performance
- This can take place when models are small and data is plentiful

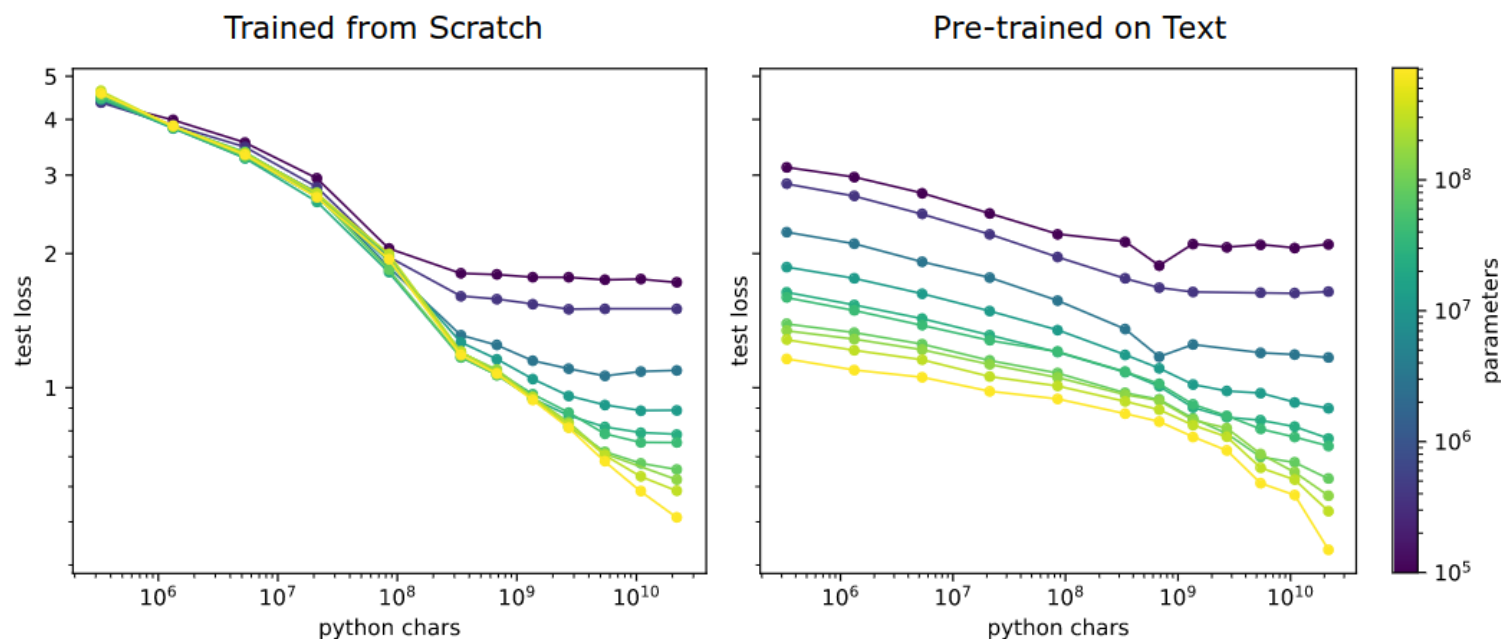


Figure 5 In the high data regime (purple lines), pre-training can effectively reduce the training set size. In other words, we get better performance training the 1M parameter models (purple) with our larger size datasets ($>1e8$) from-scratch.

Kaplan et. al. 2021

- Takeways
 - Scaling laws are pervasive in transfer as well as training
 - Pretraining is very useful when finetuning data is scarce
 - Pretraining can hurt when finetuning data is plentiful relative to model size

Emergent Abilities



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

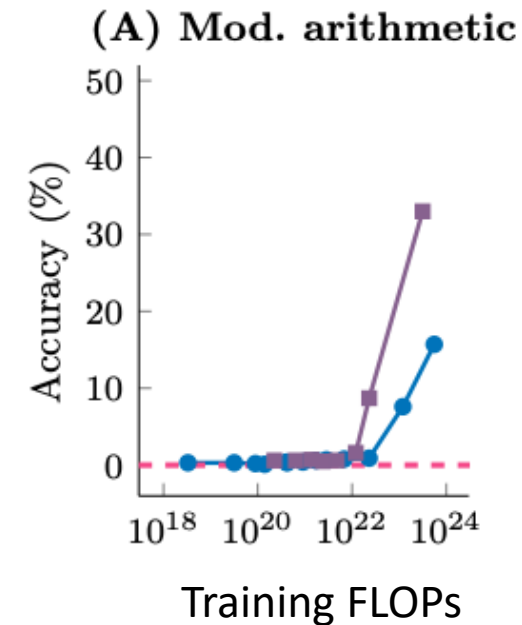
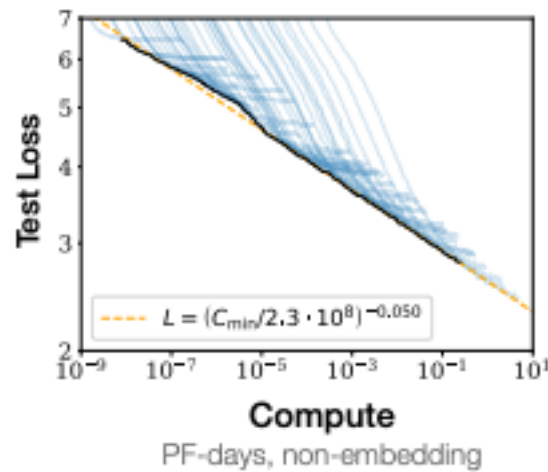
Emergent Abilities of Large Language Models, Wei et al, 2022



Main RQ: A survey and discussion of emergent abilities and their causes

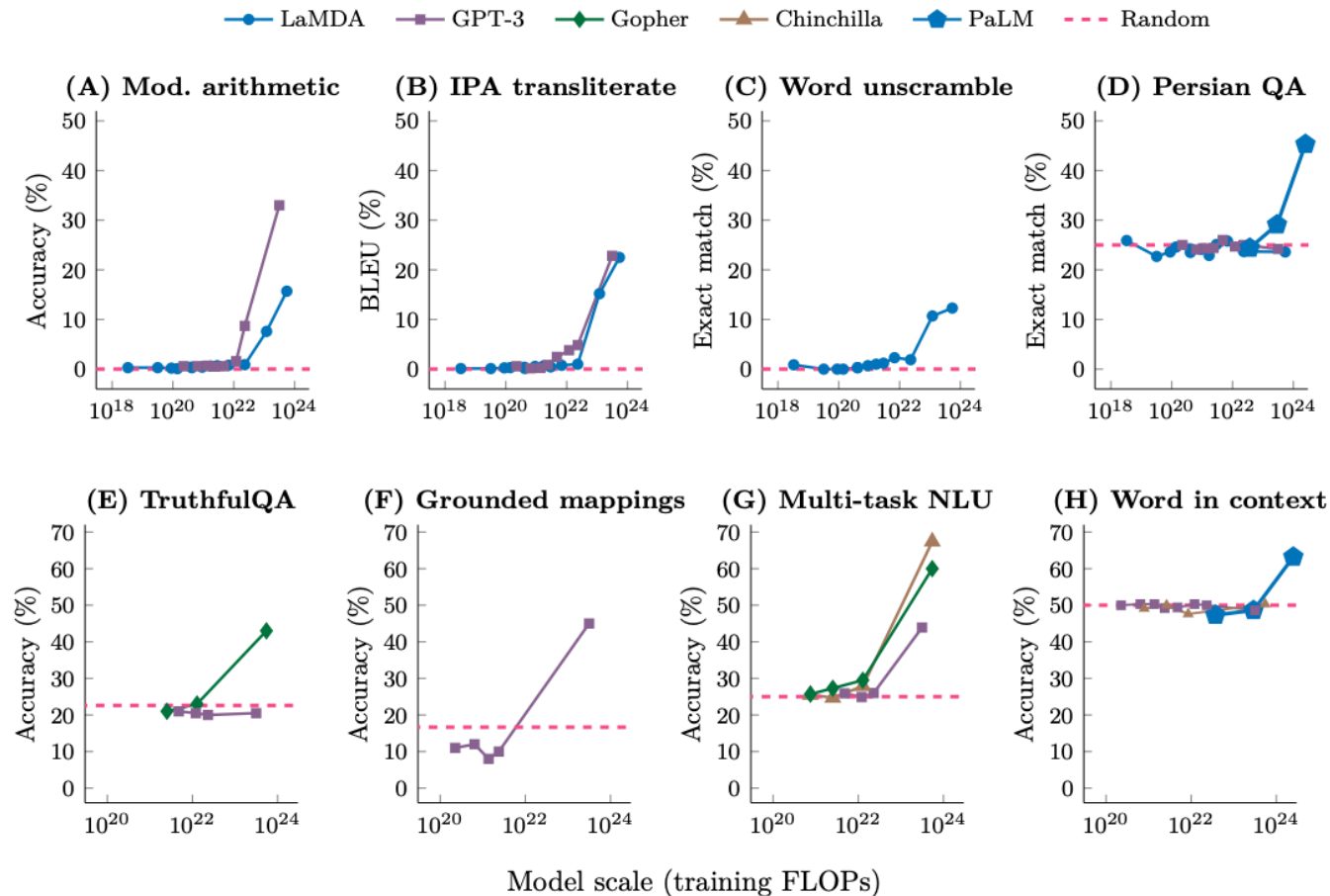
Emergent Abilities of Large Language Models, Wei et al, 2022

- Scaling Laws: A predictable and smooth relationship between scale (FLOPs or params.) and cross-entropy loss (left)
- Emergent abilities: Abilities/performance metrics that are a discontinuous function of scale (right)



Emergent Abilities of Large Language Models, Wei et al, 2022

- Examples of emergent abilities



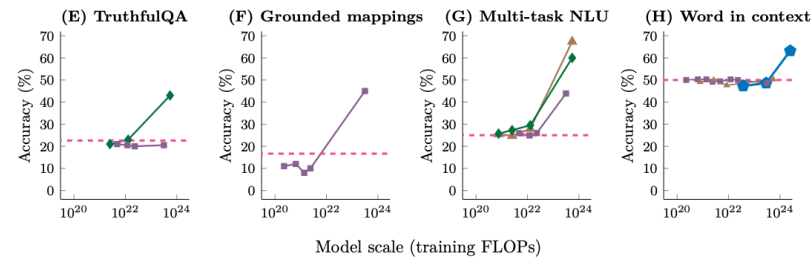
Tasks from BIG-Bench

Emergent Abilities of Large Language Models, Wei et al, 2022

- A closer look at BIG-Bench
 - Exact string matching

Target Output	Model Output	Score
Hello	laskjdfal89u09u0	0
Hello	Hellu	0
Hello	Hello	1

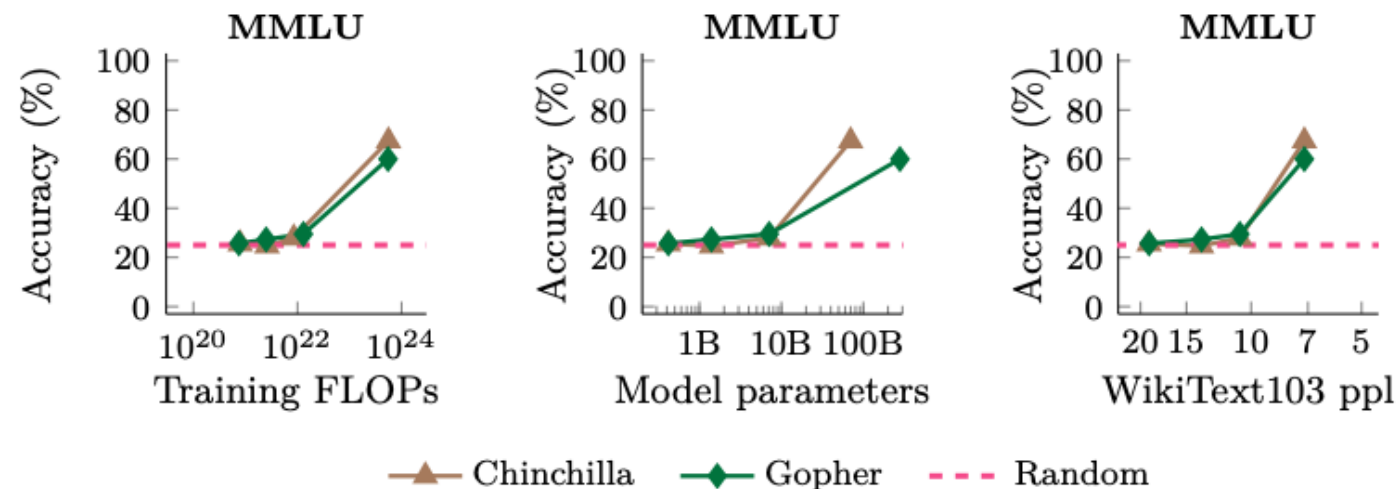
- Are the metrics hiding information? (I.e. not giving partial credit?)
- Maybe not?
 - Emergent abilities are observed on classification tasks too



Emergent Abilities of Large Language Models, Wei et al, 2022

- Perplexity (how surprised a model is by real text) may relate to emergent abilities differently

Word Index:	0	1	2	3
Word:	The	quick	brown	fox
High ppl:	0.01	0.01	0.03	0.05
Low ppl:	0.4	0.95	0.97	0.99



Emergent Abilities of Large Language Models, Wei et al, 2022

- Takeaways:
 - Overall, emergent abilities should probably be viewed as a function of many correlated variables.
 - This analysis does not explain why downstream metrics are emergent or enable us to predict the scale at which emergence occurs.
 - Even though we can predict the loss with model scale, we can't use the loss to predict what abilities a model will have

Schaeffer et. al. 2023, Are Emergent Abilities of Large Language Models a Mirage?



- **Main RQ:** Is the apparently sudden emergence of new abilities in large language models actually a result of the metrics chosen to measure them?

Schaeffer et. al. 2023 – Main Results

- Nonlinear or discontinuous metrics can appear to show sudden changes in ability where linear or continuous metrics do not
- Explains emergence on published benchmarks
- Produces apparent "emergent abilities" by changing the choice of metric

- Over 92% of emergent abilities on BIG-Bench are on tasks measured by one of the following metrics:

$$\text{Multiple Choice Grade} \stackrel{\text{def}}{=} \begin{cases} 1 & \text{if highest probability mass on correct option} \\ 0 & \text{otherwise} \end{cases}$$

$$\text{Exact String Match} \stackrel{\text{def}}{=} \begin{cases} 1 & \text{if output string exactly matches target string} \\ 0 & \text{otherwise} \end{cases}$$

- Multiple Choice Grade is discontinuous
- What about Exact String Match?

- As a theoretical example, requiring several tokens to be perfectly matched causes a sharp transition in performance:

$$\mathcal{L}_{CE}(N) = -\log \hat{p}_N(v^*) \quad \mathcal{L}_{CE}(N) = \left(\frac{N}{c}\right)^\alpha$$

$$p(\text{single token correct}) = e^{-\mathcal{L}_{CE}(N)} = e^{-(N/c)^\alpha}$$

$$\text{Accuracy}(N) \approx p_N(\text{single token correct})^{\# \text{ of tokens}} = \exp\left(-\left(\frac{N}{c}\right)^\alpha\right)^L$$

- A poor model might require many samples to observe nonzero success
- But token edit distance is not so sharp:

$$\text{Token Edit Distance}(N) \approx L(1 - p_N(\text{single token correct})) = L(1 - \exp(-(N/c)^\alpha))$$

- InstructGPT/GPT-3 family is claimed to have emergent arithmetic abilities
- Switching from Exact String Match to Token Edit Distance, these abilities emerge gradually instead of suddenly

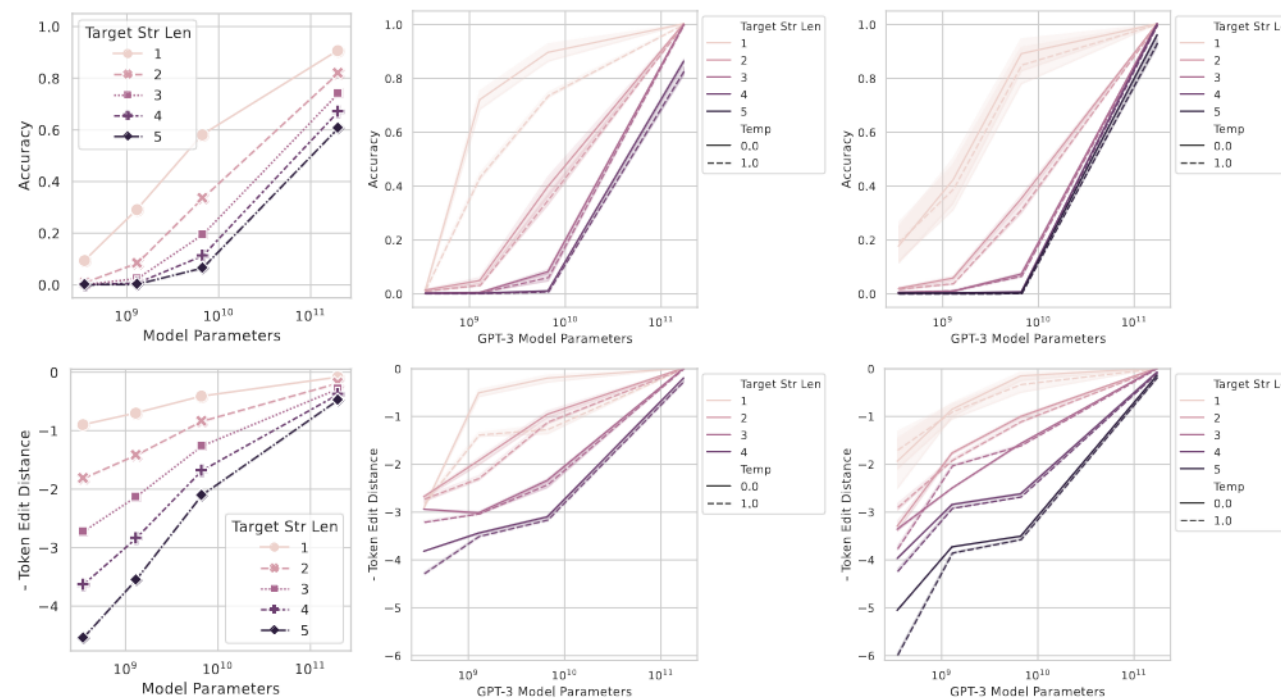


Figure 3: **Claimed emergent abilities evaporate upon changing the metric.** Left to Right: Mathematical Model, 2-Integer 2-Digit Multiplication Task, 2-Integer 4-Digit Addition Task. Top: When performance is measured by a nonlinear metric (e.g., Accuracy), the InstructGPT/GPT-3 [3, 24] family's performance appears sharp and unpredictable on longer target lengths. Bottom: When performance is instead measured by a linear metric (e.g., Token Edit Distance), the family exhibits smooth, predictable performance improvements.

- Many cases of "emergent abilities" can be attributed to the choice of metric and the resolution of measurements
- Deliberately choosing a poor metric can induce apparently emergent abilities
- Some candidate emergent abilities remain (under BLEU and normalized aggregate score metrics)

Complicating Scaling Laws



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

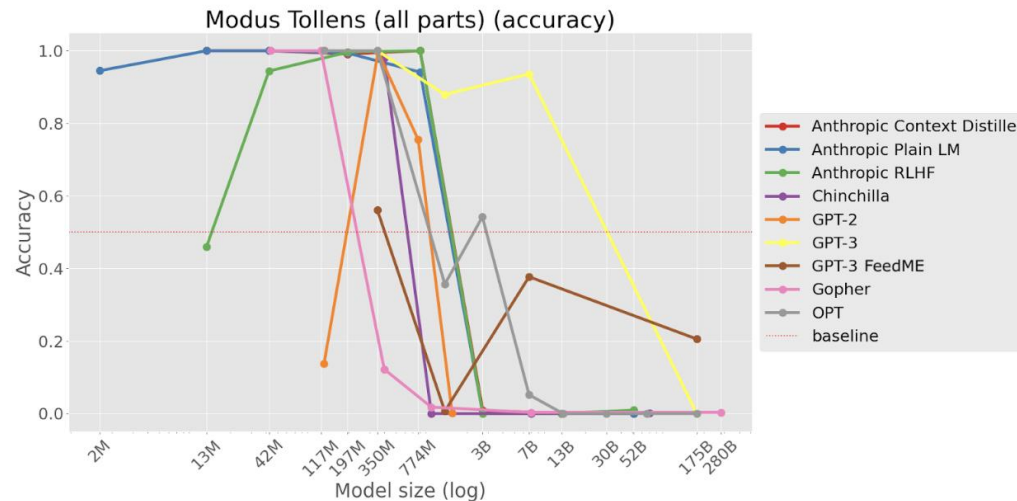
Inverse scaling can become U-shaped, Wei et al., 2022



- **Main RQ:** Over what scales does inverse scaling on certain tasks hold?

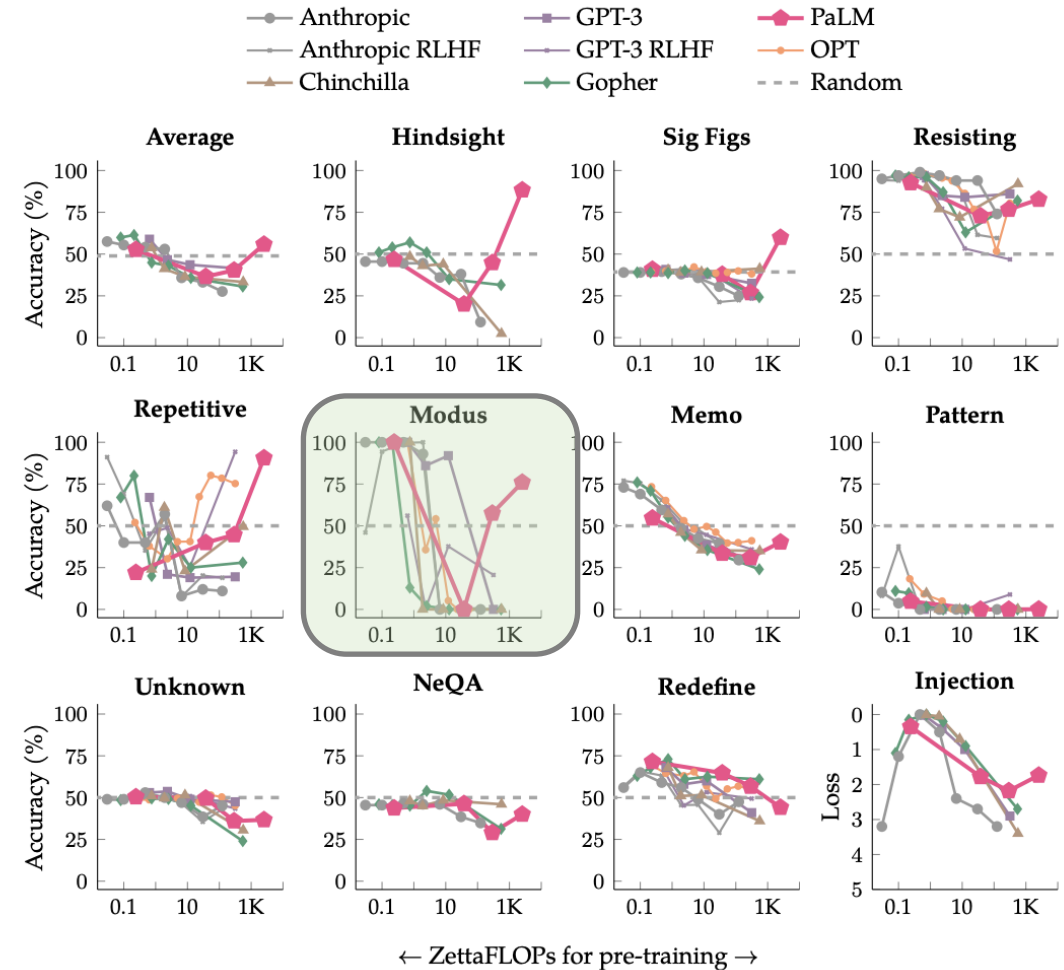
Inverse scaling can become U-shaped,
Wei et al., 2022

- Inverse Scaling Prize, McKenzie et al., 2022
 - A competition to find tasks where larger models do worse
 - 11 Inverse scaling tasks found
 - Eg. Modus tollens classification: “If p then q, not q, $\therefore$ not p”
 - Present models with a modus tollens argument and ask whether it is valid
 - Humans often make mistakes on this and larger models can better imitate these human mistakes



Inverse scaling can become U-shaped, Wei et al., 2022

Wei et al. found that
increasing size even
further will cause an
uptick in performance

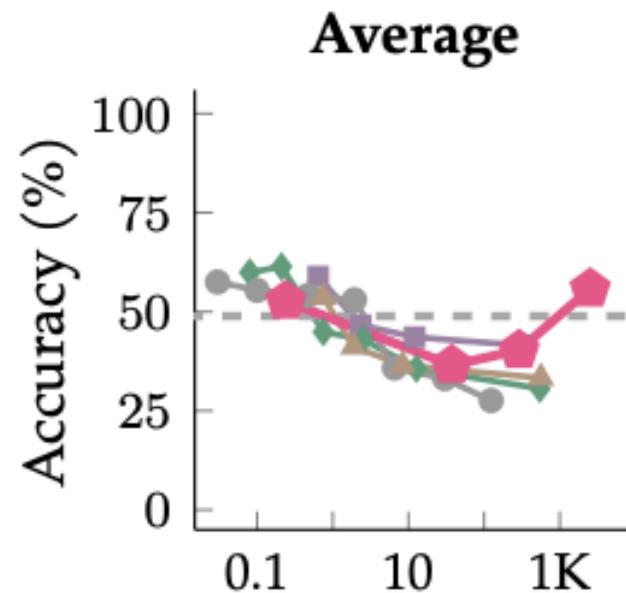


Inverse scaling can become U-shaped, Wei et al., 2022

- How to explain U-shaped scaling?
 - “We do not experimentally investigate how or why U-shaped scaling occurs, but we hypothesize that it can happen when a task contains a “distractor task”. Medium-sized models can perform the distractor task better than smaller models, which hurts performance in comparison to the smaller models. As the models scale further, the larger models can ignore the distractor task and perform the true task, which can be seen as an emergent ability that derives from scaling”
 - E.g.. For modus tollens
 - Distractor task: Produce most likely answer (and replicate common human errors)
 - True task: Perform valid logical reasoning

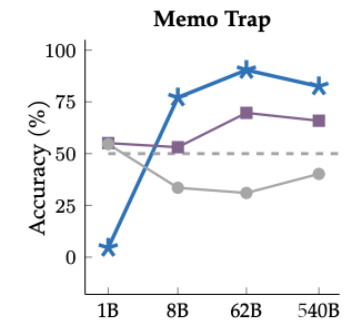
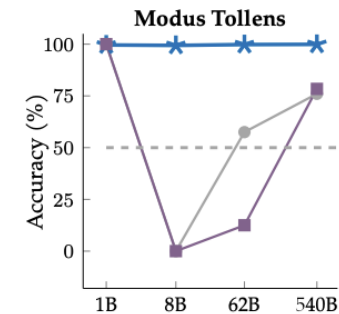
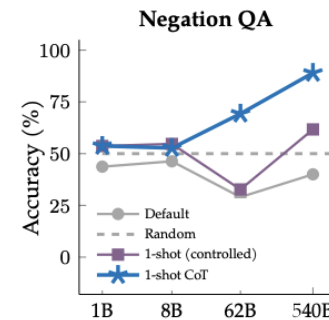
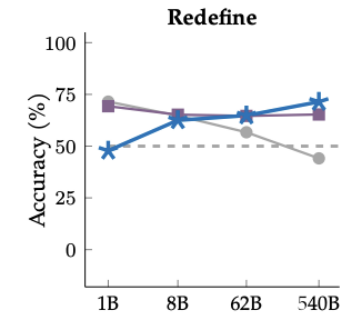
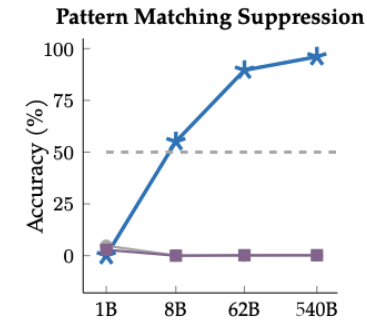
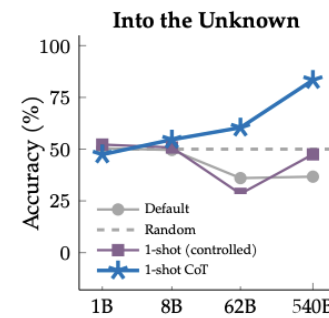
Inverse scaling can become U-shaped, Wei et al., 2022

- However, even though large models perform better than medium models, they only perform about as well as small models
- Inverse scaling tasks are not solved



Inverse scaling can become U-shaped, Wei et al., 2022

- How to improve?
 - 1-shot
 - Chain of thought (CoT)



← Model scale (# params) →

Inverse scaling can become U-shaped, Wei et al., 2022



- Takeaways
 - Increasing scale seems to turn inverse scaling into U-shaped scaling
 - Inverse scaling may not be a big issue
 - However, there is still room for improvement
 - Techniques like 1-shot prompting and CoT can help

Transcending Scaling Laws with 0.1% Extra Compute, Tay et al., 2022



Main RQ: Can new training techniques improve scaling?

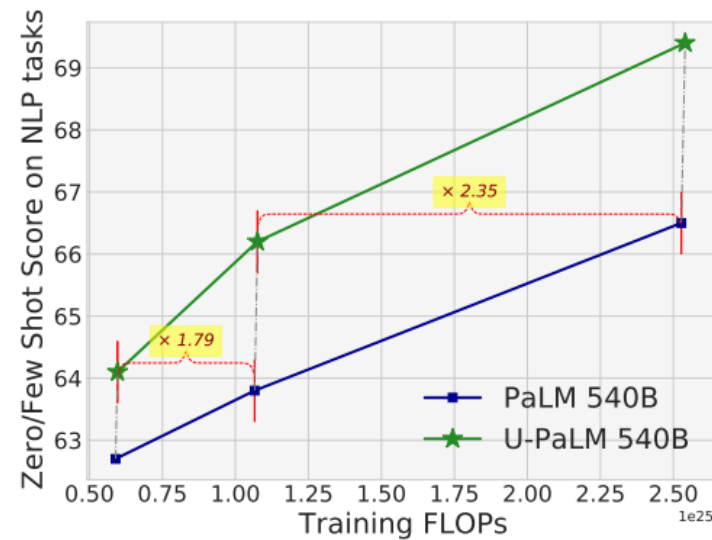
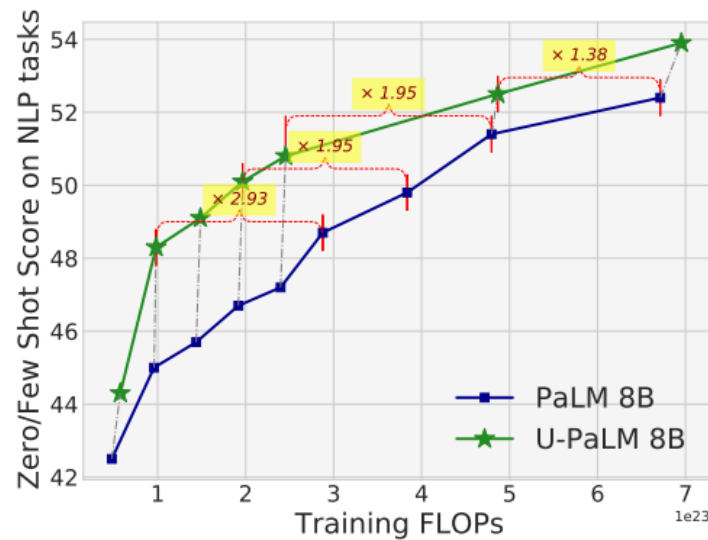
Transcending Scaling Laws with 0.1% Extra Compute, Tay et al., 2022



- The new training procedure:
 - First take the PALM base model, then
 - Use an extra 0.1% compute on the same data to train with an objective that combines prefix language modelling and long-short span corruption (infilling)
 - The quick brown fox <targets> **AND** The <targets> fox jumped over the lazy dog
 - Mixture of denoisers
 - **Regular denoising:** Noise is sampled as spans and replaced with sentinel tokens. Spans are typically uniformly sampled with a mean of 3 and a corruption rate of 15%.
 - **Extreme Denoising:** Here, the noise level is significantly increased, either by corrupting a large percentage of the original text or by replacing very long spans of text. Spans for extreme denoising are typically uniformly sampled with a mean length of 32 or a corruption rate of up to 50%.
 - **Sequential Denoising:** Noise is always sampled from the start of the text to a randomly sampled point in the text

Transcending Scaling Laws with 0.1% Extra Compute, Tay et al., 2022

- Results:
 - Improved scaling properties:



- Compute Savings:
 - For the 540B model, the saving rates at the middle checkpoint is approximately 2x. This is equivalent to about 4.4 million TPUv4 hours for the 540B model.
 - Savings seems to increase with the amount of causal language modelling already done

Transcending Scaling Laws with 0.1% Extra Compute, Tay et al., 2022

- Takeaways
 - Scaling laws may hold for autoregressive training
 - But, new training schemes, like UL2, could give us unexpected extra performance